Edge cases (when the answer spans two chunks) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chunking_practice.py` (Edge cases section): this exercise builds a document where one fact gets split across a chunk boundary, so half the fact ends up in each chunk — you create this exact failure on purpose, here, where nothing's at stake, and watch the model answer confidently from only half the information. Seeing it once means that the first time it happens for real, in the Build Task's knowledge base, you recognize it immediately instead of trusting a confident, half-right answer. **If not:** "the model gave a plausible but incomplete answer" would look identical to "the model doesn't know," and you'd have no idea which one you were debugging.

Solution

```
# chunking_practice.py — Edge cases section
import math
from openai import OpenAI

client = OpenAI()

DOCUMENT: str = (
    "The quarterly meeting will be held at 3 PM on Friday in Room 204, "
    "and everyone from the product and engineering teams is expected "
    "to attend without exception."
)
CHUNK_SIZE: int = 80
QUESTION: str = (
    "What time and room is the quarterly meeting, and who is expected "
    "to attend?"
)

def chunk_text(text: str, chunk_size: int) -> list[str]:
    results = []
    for i in range(0, len(text), chunk_size):
        results.append(text[i:i + chunk_size])
    return results

def get_embedding(text: str) -> list[float]:
```

```
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

def search(
    question: str,
    chunks: list[str],
    chunk_embeddings: list[list[float]],
    k: int,
) -> list[str]:
    question_embedding = get_embedding(question)
    scored = []
    for chunk, embedding in zip(chunks, chunk_embeddings):
        score = cosine_similarity(question_embedding, embedding)
        scored.append((score, chunk))
    # how: tuples sort by their first item — the score — highest first
    scored.sort(reverse=True)
    top_chunks = []
    for score, chunk in scored[:k]:
        top_chunks.append(chunk)
    return top_chunks

# Cut the document into fixed-size chunks and see where the facts landed.
chunks = chunk_text(DOCUMENT, CHUNK_SIZE)
print(f"document was cut into {len(chunks)} chunks:")
for i, chunk in enumerate(chunks):
    print(f"  {i}: {chunk!r}")

# Embed each chunk once.
chunk_embeddings = []
for chunk in chunks:
    chunk_embeddings.append(get_embedding(chunk))
```

```
# Compare retrieving just the top chunk (k=1) vs all of them.
top_1 = search(QUESTION, chunks, chunk_embeddings, k=1)
top_all = search(QUESTION, chunks, chunk_embeddings, k=len(chunks))

print("\nk=1 result:")
for chunk in top_1:
    print(" ", chunk)
print(f"\nk={len(chunks)} result (all chunks):")
for chunk in top_all:
    print(" ", chunk)
```